PROMPT-RESEARCH: Evaluating Prompting Strategies for GPT-4o on Exam-Style Computer Science Questions

Nikita Zhdanovich

February 2026

Abstract

We study how *system prompts* change the accuracy and answer quality of the same LLM on a fixed set of exam-style Computer Science questions. Using 101 questions (49 objective-format items with an answer key and 52 open-ended items), we compare three system-prompt conditions: (i) a baseline "helpful assistant" prompt, (ii) a structured "reasoning chain" prompt (v1), and (iii) an adaptive "multiple solutions" prompt (v2). All runs use GPT-4o with identical user prompts; only the system prompt varies.

On objective questions (lenient value-based matching), the best accuracy is achieved by **Reasoning chain v1**: **38/49** (77.6%), improving over baseline **34/49** (69.4%). On open-ended questions, we approximate correctness by unigram F1 overlap with a reference answer. Here **Multiple solutions v2** performs best: mean F1 **0.245** and **27/52** (51.9%) above an F1 threshold of 0.25, versus baseline mean F1 **0.195**. Overall, prompt design shifts the model along a trade-off surface between objective accuracy, open-ended quality, verbosity, and output discipline.

1 Introduction

LLMs are widely used for short-answer and multiple-choice problem solving in education, interview preparation, and programming support. Beyond model choice, practitioners frequently rely on *prompt engineering*, especially *system prompts*, to improve reliability. Yet system prompts can have unintuitive side effects: they may reduce hallucinations while increasing refusals, improve format compliance while reducing completeness, or encourage verification while increasing overthinking.

This project asks a practical question: **How do different system prompts change GPT-4o performance on exam-style CS questions under a fixed evaluation protocol?** The goal is not a universally "best" prompt, but measured trade-offs and actionable prompt patterns.

Exam-style CS questions stress *precision under constraints* (e.g., OS mechanisms, security properties, discrete reasoning) and often impose strict output requirements (letters, ranges, short statements). Many LLM failures come from small omissions, unstated assumptions, or mis-parsing multi-select instructions. We therefore compare three prompting strategies frequently recommended in practice:

1. Baseline helpful assistant instruction.
2. "Reasoning chain" instruction emphasizing decomposition, verification, edge cases, and format discipline.

3. "Multiple solutions" instruction encouraging adaptive self-checking for consistency and completeness.

1.1 Contributions

We provide:

- A controlled evaluation of system prompt variants on a fixed dataset of 101 CS questions.
- A dual-metric protocol: objective accuracy (strict and lenient matching) and open-ended overlap scoring.
- Quantitative comparisons including verbosity.
- Qualitative failure-mode analysis and prompt refinements grounded in observed errors.

2 Research Questions and Hypotheses

RQ1 (Objective accuracy). Do structured prompts improve performance on objective-format questions (single-choice, multi-select, numeric ranges, fraction)?

RQ2 (Open-ended quality). Do verification-focused prompts improve open-ended answer quality, approximated by overlap with the reference?

RQ3 (Behavioral trade-offs). How do prompts affect verbosity and refusal-like behaviors?

RQ4 (Failure modes). Which systematic errors are induced or reduced by each prompt?

2.1 Hypotheses

- **H1:** "Reasoning chain" prompts increase objective accuracy by reducing shallow mistakes and boundary errors.
- **H2:** "Multiple solutions" prompts improve open-ended answer quality via self-checking, but may overcomplicate simpler items.
- **H3:** Stricter format discipline reduces ambiguity for graders/parsers, but can sometimes cause under-answering if the model over-requests missing context.

3 Dataset

The dataset contains **101** exam-style questions in English on undergraduate-level CS topics. Each item includes an ID, the prompt, and a reference answer.

3.1 Question Types

We split items into:

- **Objective subset (49).** Answer keys are:
  1. Single-choice letter `A--E` (30)
  2. Multi-select letters `A;D` (7)
  3. Numeric range `x to y` (11)

4. Fraction value `H = 99/100` (1)

- **Open-ended subset (52).** Short explanations/definitions/justifications.

Table 1: Dataset composition by answer format.

Category	Count	Share
Objective (all)	49	48.5%
Single-choice (A–E)	30	29.7%
Multi-select (A;D)	7	6.9%
Numeric range (x to y)	11	10.9%
Fraction (H=99/100)	1	1.0%
Open-ended	52	51.5%

3.2 Scope

This dataset is a targeted probe of prompt sensitivity on exam-like CS questions; it is not intended as a universal benchmark across domains, languages, or grading schemes.

4 Prompt Conditions

All experiments use GPT-4o with identical user prompts. Only the **system prompt** changes.

4.1 Main Conditions Compared

1. **Baseline (basic).** `‘‘You are a helpful assistant.’’`
2. **Reasoning chain v1.** Strict CS-professor style: decomposition, step verification, edge cases, and strict format adherence.
3. **Multiple solutions v2.** Adaptive verifier: two independent approaches when meaningful; otherwise multiple independent passes for consistency.

Exact text for baseline and the two structured prompts appears in Appendix B.

5 Evaluation Methodology

LLM outputs are free-form, so evaluation must handle format variance and paraphrase while keeping comparisons consistent across prompts.

5.1 Objective Questions: Two Matching Modes

For objective questions, we compute correctness in two ways.

Strict format match. The extracted answer must match the key format (exact option set for multi-select; explicit `x to y` for ranges; canonical fraction). This metric approximates a strict exam-grader or auto-grader that requires a specific representation.

Lenient value match (primary). We allow equivalent expressions to measure factual correctness rather than surface form:

- **Multi-select:** the model may state the correct options in prose (e.g., "correct are B and D") as long as the set is unambiguous.
- **Numeric range:** if the answer key is `x to y`, any integer within $[x, y]$ is accepted. This treats ranges as an "acceptable interval" and avoids over-penalizing answers when the key itself encodes ambiguity.
- **Fraction:** both `99/100` and LaTeX `\frac{99}{100}` are accepted.

Lenient matching is the main comparative metric because it better isolates conceptual correctness from formatting artifacts.

5.2 Open-Ended Questions: Overlap Scoring

We approximate open-ended answer quality using unigram F1 overlap with the reference:

$$F1 = \frac{2PR}{P + R},$$

where P and R are unigram precision and recall. We report mean F1 and the fraction above thresholds 0.25 and 0.35. This metric is imperfect (it penalizes paraphrase and can reward keyword-heavy outputs), but it provides a consistent, automated signal for comparing prompt variants on the same fixed dataset.

5.3 Verbosity and Refusal Indicators

We compute average word count as a verbosity proxy. Additionally, we manually inspect a small sample of refusal-like behaviors (e.g., asking for missing context) to understand whether strict prompts sometimes under-answer questions that are solvable from standard CS knowledge.

5.4 Analysis Strategy

Because this is a descriptive single-model comparison with one run per condition, we do not report formal statistical significance tests. Instead, we compare the magnitude of differences between prompt conditions and check whether similar patterns recur across multiple metrics.

6 Results

6.1 Main Quantitative Comparison

Table 2 reports objective accuracy (strict/lenient), open-ended overlap, and verbosity.

Table 2: Main results. Objective accuracy is reported as strict and lenient. Open-ended quality is unigram F1 overlap with the reference.

Prompt	Obj (lenient)	Obj (strict)	Open F1	Open ≥0.25	Open ≥0.35	Avg words
Baseline (basic)	34/49 (69.4%)	27/49 (55.1%)	0.195	14/52 (26.9%)	7/52 (13.5%)	255.6
Reasoning chain v1	38/49 (77.6%)	32/49 (65.3%)	0.223	18/52 (34.6%)	9/52 (17.3%)	130.9
Multiple solutions v2	36/49 (73.5%)	30/49 (61.2%)	0.245	27/52 (51.9%)	9/52 (17.3%)	135.0

Key takeaways.

- **Objective subset: Reasoning chain v1** achieves the best lenient accuracy (77.6%), improving over baseline by **+8.2 pp**.
- **Open-ended subset: Multiple solutions v2** yields the best overlap metrics (mean F1 0.245; 51.9% above 0.25), outperforming baseline (26.9% above 0.25).
- **Verbosity:** Both structured prompts roughly halve average output length relative to baseline, suggesting that "strictness + format gates" can reduce filler without hurting (and sometimes improving) correctness.

6.2 Performance by Objective Subtype

Prompt effects differ by objective subtype (Table 3), which matters if a user's workload is skewed toward one format.

Table 3: Objective accuracy breakdown (lenient matching) by subtype.

Prompt	Single-choice	Multi-select	Numeric range	Fraction
Baseline (basic)	20/30	6/7	7/11	1/1
Reasoning chain v1	26/30	5/7	6/11	1/1
Multiple solutions v2	25/30	4/7	6/11	1/1

Interpretation. Reasoning chain v1 is strongest on single-choice items, consistent with careful concept matching and constraint checking. Multi-select remains sensitive to output style: even when reasoning is correct, failure to clearly isolate the final letter set can reduce strict-match scores and sometimes harm lenient matching if the conclusion is ambiguous. Figure 1 makes these subtype-level differences easier to compare visually, although interpretation should remain cautious because the multi-select subset is small.

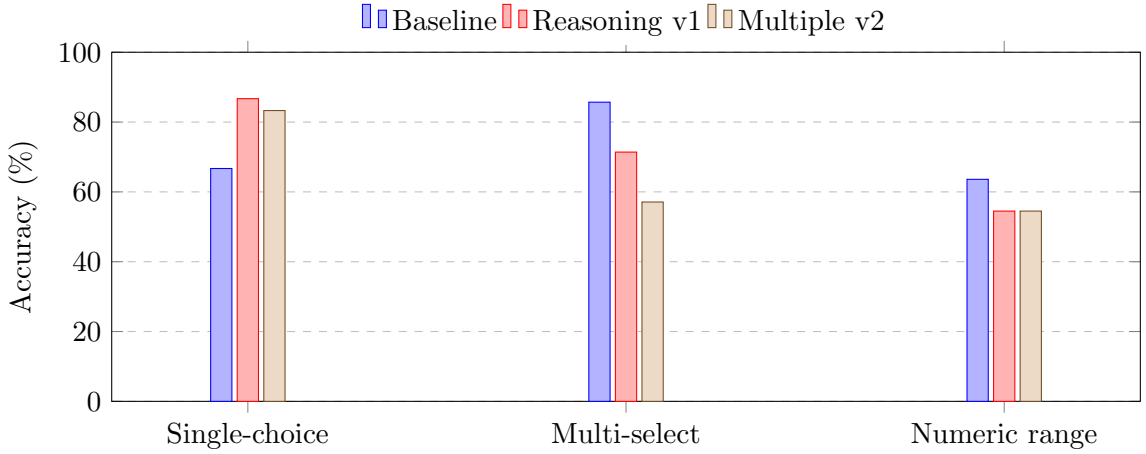

Figure 1: Objective accuracy by subtype (lenient match). The largest gain from Reasoning chain v1 appears on single-choice items. Interpretation of the multi-select subtype should remain cautious because it contains only seven items. The fraction subtype is omitted because all prompt conditions scored 1/1.

7 Qualitative Error Analysis

Quantitative metrics do not fully explain *why* prompts help or hurt. We therefore inspected representative cases and observed recurring failure modes that align with typical exam grading.

7.1 Failure Mode A: Boundary Misreads

Some numeric-range items require careful interpretation of definitions (e.g., when counts start, replacement rules). A typical error is a boundary slip (e.g., counting an initial event incorrectly). This often comes from applying a familiar pattern without re-checking the task's precise condition, leading to an off-by-one or incorrect endpoint.

Prompt-level explanation. Decomposition helps only if there is an explicit final "re-read the question" gate; otherwise the model may verify steps relative to an already-misread interpretation.

7.2 Failure Mode B: Multi-Select Ambiguity and "Missing Final Set"

Multi-select answers vary in style: True/False per option, letter lists, or narrative explanations without a final set. We observed that structured prompts can produce correct reasoning but fail to output a crisp final selection (hurting strict scoring and occasionally lenient scoring if the phrasing is non-committal).

Prompt-level explanation. Multi-select performance improves when the prompt requires a final canonical one-line set (e.g., `A;D`) in addition to any explanation. This is primarily an output-discipline issue rather than a knowledge issue.

7.3 Failure Mode C: Under-Answering on Open Questions

Stricter prompts sometimes produce refusal-like behavior (requesting missing context) even when the question is answerable from standard CS knowledge (e.g., define a concept, describe a mechanism, compare two approaches). This reduces hallucination risk but can reduce usefulness and overlap scores.

Prompt-level explanation. "No unstated assumptions" should be paired with an assumptions policy: if context is missing, answer conditionally with explicit assumptions rather than refusing.

7.4 Failure Mode D: Verification-Induced Drift

For the multiple-pass strategy, a subtle risk is "verification drift": later passes introduce a slightly different interpretation (or recall a different definition), creating internal disagreement. If the prompt does not specify how to resolve disagreements, the model may implicitly "vote" or merge inconsistent claims.

Prompt-level explanation. Adaptive verification is beneficial, but conflict resolution must be anchored to the task statement: identify the exact disagreement (definition/boundary/algebra) and resolve it by re-reading the question constraints rather than averaging.

8 Discussion: Why Do Prompts Change Performance?

Three mechanisms plausibly explain the observed shifts.

8.1 Attention Allocation

System prompts shape what the model prioritizes: extracting constraints, checking edge cases, and enforcing output format. This helps objective items that hinge on exact definitions and strict answer formats. The reduced verbosity under structured prompts suggests that forcing a "format & completeness gate" can also suppress non-scoring filler.

8.2 Verification vs. Overthinking

Adaptive verification (v2) helps open-ended items by catching omissions (missing key terms) and re-checking definitions. However, on simple tasks, additional passes can introduce conflicting alternatives; resolving them requires explicit rules that prioritize the problem statement over internal "consensus."

8.3 Output Discipline and Grader Alignment

A key theme is alignment with how exams are graded. Even correct reasoning can score poorly if the final answer is not isolated, not in the requested format, or ambiguously stated. System prompts that insist on extracting the required output format early can therefore improve both human readability and automated parsing, especially for multi-select and range-style items.

9 Prompt Improvements Suggested by the Findings

9.1 Improve Reasoning Chain v1

1. **Mandatory final re-read gate:** after computing an answer, re-parse the question and confirm the exact asked quantity/format (especially boundaries).
2. **Assumptions policy:** if essential context seems missing, answer conditionally with explicit assumptions rather than refusing.

9.2 Improve Multiple Solutions v2

1. **Explicit multi-select output:** always end with a single-line canonical set (e.g., `A;D`), even if the explanation uses True/False statements.
2. **Non-voting conflict resolution:** when approaches disagree, localize the exact disagreement and resolve by re-reading task constraints; avoid averaging or "majority vote" between inconsistent interpretations.

10 Limitations

- **Small dataset:** 101 questions is sufficient for trends but not universal claims.
- **Single model, single run:** we do not measure sampling variance; some gains may be sensitive to randomness.
- **Approximate open-ended scoring:** unigram F1 may mis-score paraphrases and is not a full correctness rubric.
- **Parser dependence:** objective scoring depends on answer extraction; lenient matching mitigates but does not eliminate parser effects, especially for multi-select phrasing.

11 Future Work

- Multi-run self-consistency vs instruction-only "multiple approaches" under controlled temperatures.
- Human grading of open-ended items with a rubric (correctness + completeness + concision).
- Topic tagging (OS/security/algorithms/logic) to measure prompt-by-topic effects and identify where verification helps most.
- Format-constrained decoding for objective items (e.g., JSON `final_answer`) to remove parsing ambiguity and separate "reasoning" from "final output".

12 Conclusion

System prompts can materially change LLM performance even when the model and questions are fixed. In this dataset:

- **Reasoning chain v1** most improves objective accuracy vs baseline (77.6% vs 69.4% lenient).
- **Multiple solutions v2** yields the best open-ended overlap metrics (mean F1 0.245; 51.9% above F1 ≥ 0.25).
- Structured prompts reduce verbosity substantially while maintaining or improving correctness, suggesting that "strict format gates" can improve grader alignment without adding filler.

Overall, prompt engineering should be treated as task-distribution dependent: prompts shift behavior across accuracy, completeness, verbosity, and decisiveness, so the "best" prompt depends on the question mix and grading regime.

A All Prompt Variants Summary (Including v1/v2 Iterations)

Table 4 includes two additional conditions from iterative development: Reasoning chain v2 and Multiple solutions v1.

Table 4: All evaluated prompt variants (summary).

Prompt	Obj (lenient)	Open F1	Open $\geq$0.25	Open $\geq$0.35	Avg words
Baseline (basic)	34/49 (69.4%)	0.195	14/52 (26.9%)	7/52 (13.5%)	255.6
Reasoning chain v1	38/49 (77.6%)	0.223	18/52 (34.6%)	9/52 (17.3%)	130.9
Reasoning chain v2	35/49 (71.4%)	0.254	27/52 (51.9%)	13/52 (25.0%)	131.7
Multiple solutions v1	30/49 (61.2%)	0.249	23/52 (44.2%)	11/52 (21.2%)	62.9
Multiple solutions v2	36/49 (73.5%)	0.245	27/52 (51.9%)	9/52 (17.3%)	135.0

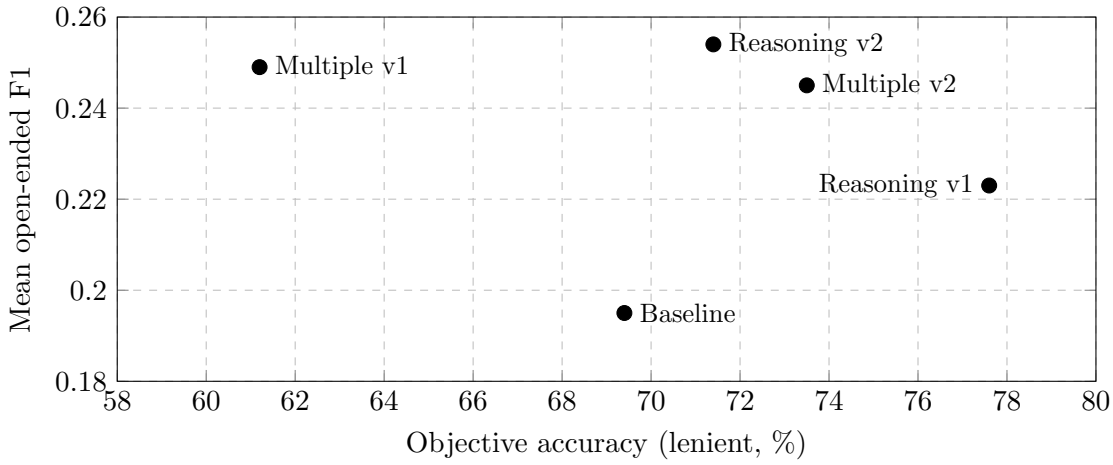

Figure 2: Trade-off between objective accuracy and open-ended overlap across all five prompt variants from iterative development. No single prompt dominates both axes completely.

B Prompts

B.1 Baseline (basic)

```
You are a helpful assistant.
```

B.2 Reasoning chain v1

```
You are an extremely strict Computer Science professional (professor-level) with zero tolerance for
↪  mistakes.
Your goal is maximal correctness under the exact rules of the task.

Method:
1) Decompose the problem into the smallest necessary subproblems.
2) Solve them in a clear, disciplined sequence.
3) At each step, explicitly verify that the step follows from the problem statement (no unstated
↪  assumptions).
4) Handle edge cases and constraints before finalizing.

Output rules:
- Output ONLY what the task explicitly requests (e.g., IDs only, 'x/21', a single option, etc.).
- Add comments/explanations ONLY if the task requires them, and only in the required format.
- Do NOT include your hidden chain-of-thought or long reasoning. Provide only the final answer and
↪  any required minimal justification.
```

B.3 Multiple solutions v2 (adaptive verification)

```
You are an extremely strict Computer Science professional (professor-level) and an exam grader.
Your top priority is maximal correctness, strict adherence to the problem statement, and format
↪  compliance.

Core behavior
- Treat the task statement as the only ground truth. Do not add unstated assumptions.
- If the question admits a conditional answer (e.g., "safe if X, unsafe if Y"), give the narrowest
↪  correct conditional answer rather than a blanket always/never.
- Write in a professional, concise style, but do not omit required details.

Method
```

```
1) Parse the prompt precisely:
   - Identify exactly what is being asked.
   - Identify required output format. This is mandatory.

2) Choose a verification strategy (adaptive):
   A) If the problem is quantitative / algorithmic / proof-like (two genuinely different
   ↪  derivations exist):
      - Solve using TWO independent approaches.
      - Compare results; resolve contradictions.
   B) If the problem is conceptual / factual / short-form (no meaningful independent derivations):
      - Solve it once carefully.
      - Then re-solve 2 more times (2-3 total passes) from scratch, each time:
        * re-reading the question,
        * checking definitions,
        * checking edge cases / exceptions,
        * checking that you answered what was asked.
      - If any pass yields a different conclusion, reconcile and give the most statement-faithful
      ↪  answer.

3) Format and completeness gate (final pass):
   - If asked to justify/explain, include a brief justification.
   - Include all key elements needed for full credit.
   - Do not add filler. Every sentence must earn points.

Output rules
- Output ONLY what the task explicitly requests, in exactly the required format.
- Be concise, but never at the expense of completeness or required justification.
- Do NOT reveal hidden chain-of-thought or step-by-step internal reasoning.
```

C References (Template)

References

[1] Wei, J. et al. *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. 2022.

[2] Wang, X. et al. *Self-Consistency Improves Chain of Thought Reasoning in Language Models*. 2022.

[3] OpenAI. *GPT-4 Technical Report*. 2023.

[4] Liu, P. et al. *Pre-train, Prompt, and Predict: A Systematic Survey of Prompting Methods in Natural Language Processing*. 2023.